

Šķēršļi lamai

Lama vēlas ceļot pāri Altiplano plakankalnei. Lamai ir pieejama plakankalnes karte režģa formā, kas sastāv no $N \times M$ kvadrātveida rūtiņām. Kartes rindas ir numurētas no 0 līdz $N - 1$ no augšas uz leju, un kolonnas ir numurētas no 0 līdz $M - 1$ no kreisās puses uz labo. Kartes rūtiņu i -tajā rindā un j -tajā kolonnā ($0 \leq i < N, 0 \leq j < M$) apzīmēsim ar (i, j) .

Lama ir izpētījusi plakankalnes klimatu un secinājusi, ka visām rūtiņām, kas atrodas katrā kartes rindā, ir viena un tā pati **temperatūra** un visām rūtiņām, kas atrodas katrā kartes kolonnā, ir viens un tas pats **gaisa mitrums**. Lama Jums ir iedevusi divus veselu skaitļu masīvus T un H attiecīgi garumā N un M . Šeit $T[i]$ ($0 \leq i < N$) norāda temperatūru rūtiņām i -tajā rindā, un $H[j]$ ($0 \leq j < M$) norāda gaisa mitrumu rūtiņām j -tajā kolonnā.

Lama ir izpētījusi arī plakankalnes floru un pamanījusi, ka rūtiņa (i, j) **nesatur veģetāciju** tad un tikai tad, ja tās temperatūra ir lielāka par tās gaisa mitrumu, formāli $T[i] > H[j]$.

Lama var ceļot pāri plakankalnei, tikai sekojot **derīgiem ceļiem**. Derīgs ceļš ir atšķirīgu rūtiņu virkne, kas apmierina šādus nosacījumus:

- Ceļā katram secīgu rūtiņu pārim ir kopīga mala.
- Visas ceļa rūtiņas nesatur veģetāciju.

Jūsu uzdevums ir atbildēt uz Q vaicājumiem. Katram vaicājumam Jums ir doti četri veseli skaitļi: L, R, S un D . Jums jānosaka, vai eksistē derīgs ceļš, kuram izpildās:

- Ceļš sākas rūtiņā $(0, S)$ un beidzas rūtiņā $(0, D)$.
- Visas rūtiņas ceļā atrodas tikai starp L -to un R -to kolonnu, ieskaitot.

Ir garantēts, ka gan $(0, S)$, gan $(0, D)$ nesatur veģetāciju.

Implementācijas detaļas

Pirmā procedūra, kas Jums jāimplementē, ir:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : masīvs garumā N , kas norāda temperatūru katrā rindā.
- H : masīvs garumā M , kas norāda gaisa mitrumu katrā kolonnā.

- Šī procedūra katram testam tiek izsaukta tieši vienu reizi, pirms tiek veikts kaut viens `can_reach` izsaukums.

Otrā procedūra, kas Jums jāimplementē, ir:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : veseli skaitļi, kas apraksta vaicājumu.
- Šī procedūra tiek izsaukta Q reizes katram testam.

Šai procedūrai jāatgriež `true` tad un tikai tad, ja eksistē derīgs ceļš no rūtiņas $(0, S)$ uz rūtiņu $(0, D)$, tāds ka visas rūtiņas ceļā atrodas tikai starp L -to un R -to kolonnu, ieskaitot.

Ierobežojumi

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ katram i , kur $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ katram j , kur $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Abas rūtiņas $(0, S)$ un $(0, D)$ nesatur veģetāciju.

Apakšuzdevumi

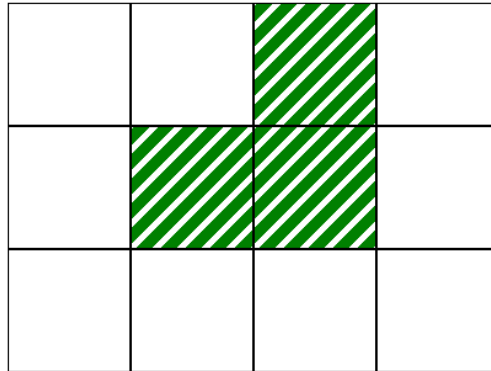
Apakšuzdevums	Punkti	Papildu ierobežojumi
1	10	$L = 0, R = M - 1$ katram vaicājumam. $N = 1$.
2	14	$L = 0, R = M - 1$ katram vaicājumam. $T[i - 1] \leq T[i]$ katram i , kur $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ katram vaicājumam. $N = 3$ un $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ katram vaicājumam. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ katram vaicājumam.
6	17	Bez papildu ierobežojumiem.

Piemērs

Aplūkosim šādu izsaukumu:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

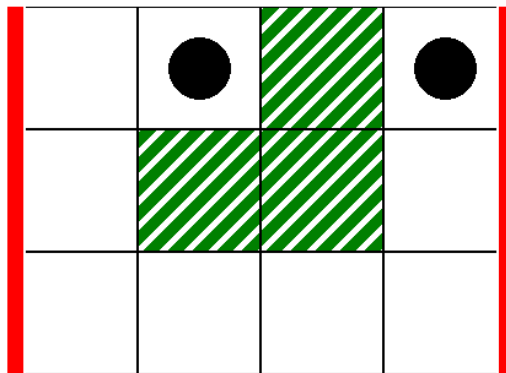
Šis atbilst tālāk attēlotajai kartei, kur baltās rūtiņas nesatur veģetāciju:



Kā pirmo vaicājumu aplūkosim šādu izsaukumu:

```
can_reach(0, 3, 1, 3)
```

Šis atbilst situācijai tālāk redzamajā attēlā, kur biezās vertikālās līnijas norāda kolonnu intervālu no $L = 0$ līdz $R = 3$ un melnie aplīši norāda sākuma un beigu rūtiņas:



Šajā gadījumā lama var no rūtiņas (0, 1) sasniegt rūtiņu (0, 3) pa šādu derīgu ceļu:

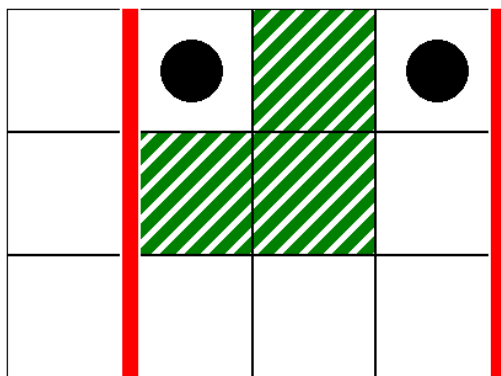
(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)

Tātad, šim izsaukumam jāatgriež `true`.

Kā otro vaicājumu aplūkosim šādu izsaukumu:

```
can_reach(1, 3, 1, 3)
```

Šis atbilst scenārijam tālāk redzamajā attēlā:



Šajā gadījumā nav derīga ceļa no rūtiņas $(0,1)$ uz rūtiņu $(0,3)$, kura visas rūtiņas atrodas tikai starp 1. un 3. kolonnu, ieskaitot. Tātad, šim izsaukumam jāatgriež `false`.

Paraugvērtētājs

Ievaddatu formāts:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Šeit $L[k]$, $R[k]$, $S[k]$ un $D[k]$ ($0 \leq k < Q$) norāda parametrus katram `can_reach` izsaukumam.

Izvaddatu formāts:

```
A[0]
A[1]
...
A[Q-1]
```

Šeit $A[k]$ ($0 \leq k < Q$) ir 1, ja izsaukums `can_reach(L[k], R[k], S[k], D[k])` atgrieza `true`, citādi 0.